libatrac3plus Overview

© 2009 Sony Computer Entertainment Inc.
All Rights Reserved.
SCE Confidential

Table of Contents

1 Overview **3**
Functional Overview and Features 3
Related Files 4
Sample Programs 4
2 Description of Operation **6**
Overview of ATRAC3™/ATRAC3plus™ Data 6
Overview of the libatrac3plus Usage Procedure 6
About On-Memory Playback and Streaming Playback 6
Loop Playback 7
Second Buffer 7
3 Usage Procedure **9**
Preparing ATRAC3™/ATRAC3plus™ Data 9
Initialization 9
Preparing Second Buffer 9
Decoding 10
Streaming Playback 10
Resetting of the Playback Position 11
Termination 12
4 Encoding Tool **13**
Overview 13
at3tool 13
Operating Environment 13
5 Precautions **14**
Supported Stream Data 14
AA3 Files 14
Output of Monaural Data 14
Streaming Playback via UMD™/DVD 14
Using libatrac3plus and the Software Audio Synthesizer (SAS) Simultaneously 15
Stack Size of Threads 15
Precautions When Using the Second Buffer 15
Termination Processing 16

1 Overview

Functional Overview and Features

The ATRAC3plus™ library (libatrac3plus) is used for controlling the ATRAC3™/ATRAC3plus™ decoder that runs on the Media Engine. The library enables ATRAC3™/ATRAC3plus™ compressed audio data that was generated using the proprietary at3tool, to be decoded to linear PCM. Also, by linking libatrac3plus with SAS (Software Audio Synthesizer), an ATRAC3™ sound source can be used as the synthesizer's phoneme region.
The following are the features of libatrac3plus.

Features

- Can decode ATRAC3™ and ATRAC3plus™ audio data to PCM.
- Supports 66 kbps, 105 kbps, and 132 kbps bit rates for ATRAC3™ (stereo).
- Supports 52 kbps and 66 kbps bit rates for ATRAC3™ (monaural).
- Supports 48 kbps, 64 kbps, 96 kbps, 128 kbps, 160 kbps, 192 kbps, 256 kbps, 320 kbps and 352 kbps bit rates for ATRAC3plus™ (stereo).
- Supports 32 kbps, 48 kbps, 64 kbps, 96 kbps and 128 kbps bit rates for ATRAC3plus™ (monaural).
- Provides loop playback function for ATRAC3™ and ATRAC3plus™ data.
- Provides streaming playback function for ATRAC3™ and ATRAC3plus™ data.

Note that libatrac3plus only provides a conversion (decoding) function for decoding from ATRAC3™/ATRAC3plus™ data to PCM data. To output decoded data to the audio output, libatrac3plus should be used together with a library such as libwave. libwave and the other sound libraries are described in the Sound Overview document. Please refer to that document for more information.
For information about at3tool, refer to the at3tool User's Manual.

Related Files

The following files are required to use libatrac3plus.

Category	Filename
Header file	libatrac3plus.h
Stub library file	libatrac3plus_stub.a
weak import stub library file	libatrac3plus_stub_weak.a

Also, a utility function must be used to load the libatrac3plus module itself. For details, refer to the Utility Overview.

Sample Programs

The following libatrac3plus sample programs are provided for reference.

(1) devkit/sample/sound/stringsound/

This sample program shows the basic method of using libatrac3plus.
It basically uses libatrac3plus to add background music to devkit/sample/graphic/string/ using libatrac3plus.

(2) devkit/sample/sound/starmix/

This sample program shows the concurrent use of libsas and libatrac3plus.
Background music is played using libatrac3plus, and at the same time, sound effects are played by libsas according to button operations.

(3) devkit/sample/sound/at3doubleplay/

This sample program plays two songs of ATRAC3™/ATRAC3plus™ data concurrently.
The code is added to the samples that are placed in sample/graphics/string, to play two sound sources of ATRAC3™/ATRAC3plus™ in a loop as BGM.

(4) devkit/sample/sound/at3streaming/

This sample program performs asynchronous read and streaming playback of ATRAC3™/ATRAC3plus™ data.
The code is added to the samples that are placed in sample/graphics/string, to play sound sources of ATRAC3™/ATRAC3plus™ in a loop as BGM.

(5) devkit/sample/sound/at3mix/

This sample program uses libsas and libatrac3plus concurrently, just like starmix.
However, sound is output using a method that has a lighter CPU load than starmix. Check the load reduction.

(6) devkit/sample/sound/at3mixEx/

This sample program uses libsndp to generate sound for at3mix sound effects.

(7) devkit/sample/sound/at3lowlevel/

This sample program uses low-level functions.

(8) devkit/sample/sound/sas-streaming/

This sample program links libatrac3plus with libsas to cause streaming playback of ATRAC3™ audio via SAS.

2 Description of Operation

Overview of ATRAC3™/ATRAC3plus™ Data

The ATRAC3™/ATRAC3plus™ data used by libatrac3plus is mainly intended for game applications. As such, the data includes additional information such as that required for playing back loops.

Overview of the libatrac3plus Usage Procedure

When libatrac3plus is used to decode ATRAC3™/ATRAC3plus™ data, a management ID (handle) known as an ATRAC™ ID is first obtained.
After that, an input buffer is associated with the ATRAC™ ID. The ATRAC™ ID can then be used to control the decoding of the desired data.
The following is a summary of the procedure for using libatrac3plus to perform processing up to the point of decoding.

(1) Prepare the input and output buffers.
(2) Obtain an ATRAC™ ID.
(3) Read the ATRAC3™/ATRAC3plus™ data into the input buffer.
(4) Associate the input buffer with the ATRAC™ ID.
(5) Using the ATRAC™ ID, output the decoded data to the output buffer.

Note that a separate ATRAC™ ID must be used for each type of codec. Also, the number of ATRAC™ IDs that can be used simultaneously is limited (by default, there are two ATRAC™ IDs for ATRAC3plus™ and two for ATRAC3™). This number can be changed by using the **sceAtracReinit**() function.

About On-Memory Playback and Streaming Playback

In libatrac3plus, an input buffer and an ATRAC™ ID are associated together to perform playback processing. If the size of the input buffer is greater than the size of the ATRAC3™/ATRAC3plus™ data, it is not necessary to read in the data after associating the buffer and ATRAC™ ID. This type of data playback method is known as on-memory playback. Conversely, if the size of the input buffer is less than the size of the ATRAC3™/ATRAC3plus™ data, processing will need to be performed to supply follow-on data. This type of playback method is known as streaming playback.

Loop Playback

As shown in Figure 1, particular sections of a tune can be played back multiple times (or endlessly) in a loop. at3tool, which is described later, can be used to set the starting and ending positions of the loop. The positions can be set by specifying individual samples. libatrac3plus will play back the loop based on this information.

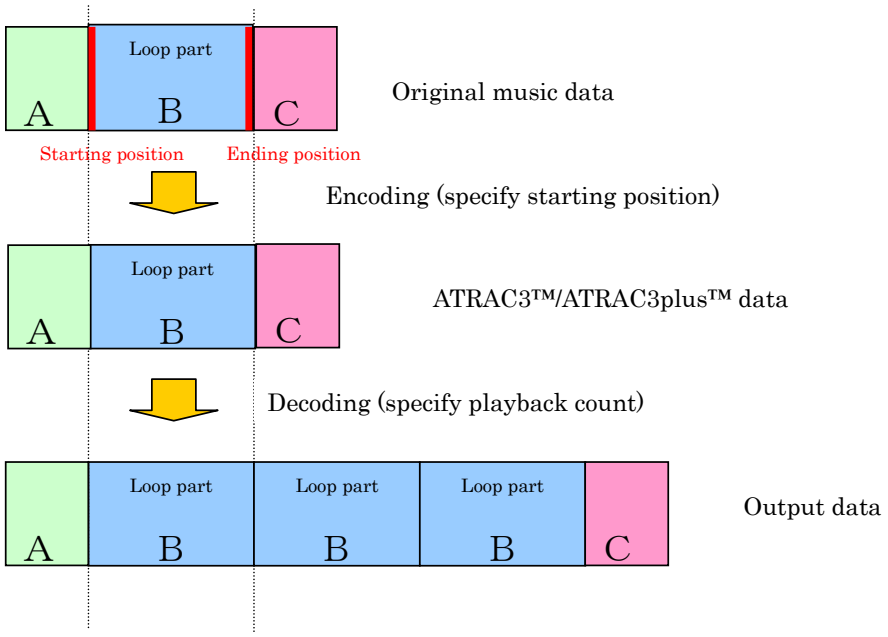

Figure 1 Overview of Loop Playback Processing

Second Buffer

Two input buffers must be prepared when performing a streaming playback of ATRAC3™/ATRAC3plus™ data that has data (C in Figure 2) after a loop processing section. The first input buffer (first buffer) is the buffer for storing the data up to and including the loop processing section, and the second input buffer (second buffer) is the buffer for storing the data after the loop processing section (see the lower section of Figure 2).

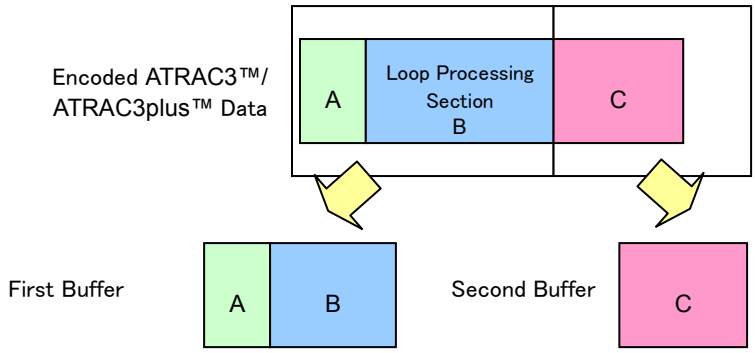

Figure 2 Buffer Control of the First Buffer and the Second Buffer

By preparing two buffers, the following will become possible (see Figure 3).
Playback can continue without causing data to be read suddenly regardless of whether the loop (B) is repeatedly played or the playback position is advanced (moved from B to C).

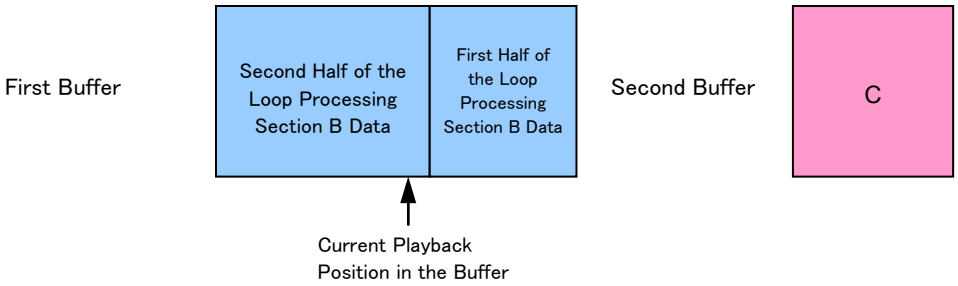

Figure 3 State of the Buffer when playing the Last Part of the Loop Processing Section B

Note

While the second buffer is playing the required data, after the playback position is moved to (C), the first buffer is still used to maintain the data in part (C).
As a result, even if the second buffer is not that large, the data read frequency does not have to increase by very much when streaming playback is being performed for the data in part (C).

3 Usage Procedure

Preparing ATRAC3™/ATRAC3plus™ Data

Use at3tool (described later) to generate ATRAC3™/ATRAC3plus™ data from PCM data (.wav file). Proper operation is not guaranteed if data generated by some other tool is used with libatrac3plus.

Initialization

Before using libatrac3plus, use the module utility to load the modules.
The modules that must be loaded are avcodec and libatrac3plus.
Then, perform initialization by using the following procedure.

(1) Call **sceAtracGetAtracID**() and obtain an ATRAC™ ID. The number of ATRAC™ IDs that can be used simultaneously is limited. If an ID cannot be obtained, release an ATRAC™ ID which is not being used.

(2) Read the ATRAC3™/ATRAC3plus™ data into the input buffer. The input buffer must be at least a certain size (several kilobytes). If the data was successfully read in, call a function such as **sceAtracSetData**() to associate the ATRAC™ ID and the input buffer. For the arguments when performing this call, specify the ATRAC™ ID obtained in step (1) in *atracID*, the address of the input buffer in *pucBufferAddr*, and the size of the input buffer in *uiBufferByte*. Doing this will cause libatrac3plus to read in the information residing in the initial portion of the input data, such as the music length and the loop information, and to initialize the variables in the library.

Preparing Second Buffer

The second buffer is needed when performing a streaming playback of ATRAC3™/ATRAC3plus™ data that has data after a loop processing section (see Chapter 2).

(1) Call the **sceAtracIsSecondBufferNeeded**() function to determine whether the second buffer is needed. If it is not needed, skip to "Decoding" in the next section.

(2) Prepare a buffer different from the input buffer used in the initialization processing as a second buffer. The second buffer must also be at least a certain size (several kilobytes).

(3) Get information about the data to be read into the second buffer using **sceAtracGetSecondBufferInfo()**. When this function is called, the offset from the beginning of ATRAC3™/ATRAC3plus™ data to the data after the loop processing section is stored in **puiPosition*, and the number of bytes from **puiPosition* to the end of the file is stored in **puiDataByte*.

(4) Based on the above information, read the data into the second buffer.

(5) After the data has been read in, call **sceAtracSetSecondBuffer()**, associate the second buffer with the ATRAC™ ID, and register the second buffer in the library.

Decoding

After the library has been initialized, use the following procedure to perform decoding.

(1) Prepare an output buffer. The size of the output buffer must be a multiple of 64 bytes, and it must be allocated on a 64-byte boundary with 64-byte alignment. The number of samples and number of channels that are obtained when decoding the next frame can be found by calling **sceAtracGetNextSample()** and **sceAtracGetOutputChannel()**. The buffer size must be at least twice the product of those values (the decoded data size of each sample is 2 bytes, or 16 bits). The maximum number of samples that can be obtained in a single decoding can be found by calling **sceAtracGetMaxSample()**.

(2) Call **sceAtracDecodeData()** with the buffer from step (1) passed in *psOutAddr*.

Streaming Playback

The following procedure should be used to perform streaming playback.

(1) After calling **sceAtracGetRemainFrame()**, check **piRemainframe* to determine how many more times **sceAtracDecodeData()** can be called and whether additional processing is needed.

(2) Call **sceAtracGetStreamDataInfo()** to find out what part of the input buffer has already been decoded and can be overwritten. Specifically, this function stores the starting address of the available area in the input buffer in **ppucWritePointer*, its size (in bytes) in **puiWritableByte*, and the offset from the beginning of the file containing the ATRAC3™/ATRAC3plus™ data where reading should continue in **puiReadPosition*.

(3) Add data to the input buffer based on the information described in (2). The amount of data that is added can be less than **puiWritableByte*, however, data must be written to the input buffer beginning at **ppucWritePointer*.

(4) Call **sceAtracAddStreamData()** to report the size of the added data to libatrac3plus.

Additional Information 1

There are many ways to determine whether additional data is needed in (1) or not. The simplest way is to divide the input buffer into two, use one of them until the end, and read additional data.
When such a method is used, it will be possible to adjust the timing for reading with the following method.
First, obtain the "number of times decoding can be performed (*N*)" in the initial state. This value can be obtained by verifying the value of **piRemainframe* after calling **sceAtracGetRemainFrame**() in a state where data has been read into the entire input buffer.
It can also be obtained by calculating according to the following equation.

$$N = \frac{Bufsize \times 8 \times 44.1}{bps \times M}$$

In this equation, *Bufsize* is the input buffer size (in units of byte), *bps* is the bit rate of ATRAC3™/ATRAC3plus™ data (in units of kbps), and *M* is the maximum number of samples to be output in a single decoding (1024 for ATRAC3™ and 2048 for ATRAC3plus™).
It is recommended to read data asynchronously or in a separate thread which has lower priority so that the processing will not be stopped by reading data.
However, due to load situations of other threads or the like, there is a possibility that the reading is not fast enough and the sound is interrupted. In such a case, it will be necessary to manage, for example, by using a bigger buffer or increasing the number of buffers to divide a buffer into.

Additional Information 2

When performing streaming playback, refer to “Streaming playback via UMD™/DVD in Chapter 5 "Precautions".

Resetting of the Playback Position

The procedure to reset the playback position is as follows.

(1) Call **sceAtracGetBufferInfoForResetting**() and use *uiSample* to specify the desired playback position.

(2) After calling the above function, check **puBufferInfo* and obtain the following information about the first buffer and second buffer.

- Start address for writing to buffer
- Maximum number of bytes which can be written to buffer
- Minimum number of bytes which must be written to buffer
- Position for reading data to be written to buffer

(3) Set the data to the first buffer and second buffer based on the above information. The number of bytes to write must be greater than or equal to the number of bytes required and less than or equal to the maximum number of bytes.

(4) Using **sceAtracResetPlayPosition**(), inform libatrac3plus of the data size which was reset above and the *uiSample* playback position.

Notes

For on-memory playback, the buffer control processing performed in steps (1) to (3) above is not necessary. Set uiWriteByteFirstBuf and uiWriteByteSecondBuf of **sceAtracResetPlayPosition**() to 0 instead.

Termination

During libatrac3plus termination processing, the ATRAC™ IDs that were allocated during initialization are released. Since only a limited number of ATRAC™ IDs can be used, be sure to release any IDs when they are not being used. The **sceAtracReleaseAtracID**() function can be used to perform this function.

4 Encoding Tool

Overview

at3tool is a tool that can be used to generate ATRAC3™/ATRAC3plus™ data specifically for libatrac3plus.

at3tool

at3tool combines an encoder/decoder for generating and verifying ATRAC3™/ATRAC3plus™ data that is used by libatrac3plus. Using at3tool to perform encoding and decoding enables the sound quality and size of the compressed data to be known in advance, and data to be generated according to a desired compression rate.

Operating Environment

The following is the recommended operating environment for at3tool.
For details, refer to the at3tool User's Manual.

- Windows XP Service Pack 3 (Japanese version and English version)
- Windows Vista Service Pack 1 (Japanese version and English version)
- Linux

5 Precautions

Supported Stream Data

Be sure to use data that was generated by at3tool and a tool that supports AA3 file output. Proper operation is not guaranteed if data generated by other tools is used with libatrac3plus.

AA3 Files

AA3 is an acronym for ATRAC™ Audio with ID3v2 tag. An AA3 file is an unencoded ATRAC™ file with no copyright protection. It consists of three blocks: a metadata block which is similar to the ID3v2 tag, an audio header block, in which codec information is written, and an audio data block containing encoded ATRAC™ data.
libatrac3plus supports only simple file playback and cannot parse the ID3v2 tag.
A tool that can generate AA3 files as of April 2006 is the ACID Pro 6, which is sold by Sony Media Software.

Output of Monaural Data

When monaural data is decoded, either monaural or stereo can be selected as the decoding result (the same data is output to both the left and right channels). You can find details on this in the section on initialization functions in the libatrac3plus reference.

Streaming Playback via UMD™/DVD

When performing streaming playback via UMD™/DVD, the **buf* pointer, the second argument of **sceIoRead**(), should conform to 4-byte alignment.
When performing streaming playback with libatrac3plus, the address for the next data is specified in **ppucWritePointer,* the second argument of **sceAtracGetStreamDataInfo**(). In order for the **ppucWritePointer* to have an address that always conforms to 4-byte alignment, the following conditions must be satisfied:

(1) The buffer set by the initialization function, such as **sceAtracSetData**(), **sceAtracSetHalfwayBuffer**(), **sceAtracSetDataAndGetID**(), or **sceAtracSetHalfwayBufferAndGetID**(), conforms with 4-byte alignment and in an area of multiples of 4-byte.

(2) *uiReadByte* (number of valid bytes that were read from the beginning of the buffer) set by **sceAtracSetHalfwayBuffer**() or **sceAtracSetHalfwayBufferAndGetID**() is mupltiples of 4-byte.

(3) The buffer set by **sceAtracSetSecondBuffer**() function conforms with 4-byte alignment and in an area of multiples of 4-byte.

(4) The number of bytes specified in *uiAddByte* for the **sceAtracAddStreamData**() function is a multiple of four.

Using libatrac3plus and the Software Audio Synthesizer (SAS) Simultaneously

The core part of libatrac3plus (the ATRAC3™/ATRAC3plus™ decoder) runs on the Media Engine. Furthermore, libsas calls SAS, which also runs on the Media Engine. As a result, when both libatrac3plus and libsas are used at the same time, there may be contention for Media Engine resources. In the worst case, SAS sound may get cut off. Please see the libsas overview (libsas-Overview) document for information on processing load when libsas and libatrac3plus are used at the same time.
Moreover, using the libatrac3plus loop-playback function may temporarily increase the load around the point where the loop returns. Therefore when using the libatrac3plus loop-playback function together with SAS, make sure that SAS output does not get cut off around the point where the loop returns.

Stack Size of Threads

sceAtracDecodeData() uses about 10 KB of stack space, which is the largest for any of the libatrac3plus functions. Therefore, when a thread that uses libatrac3plus is created, a bigger stack size than usual should be created.
To check the stack size in detail and reduce it to the minimum required, use **sceKernelGetThreadStackFreeSize**(), which is defined in thread.h.

Precautions When Using the Second Buffer

To perform streaming playback of data for which the second buffer is required (see Chapter 2), the location where data is to be read depends on the location that is currently being decoded. If A denotes the location before the loop and B denotes the part after the loop, then the following shows the location being decoded and the location where data will be read for these two cases.

When the location that is currently being decoded is A:
After the current location is read, the location of A that follows that location will be the next location read.
To support the case where the number of loops is changed after data for the number of loops that is currently set has been completely read into the buffer, the A part will be read as data for which loop playback processing will continue.

When the location that is currently being decoded is B:
After the current location is read, the part of B that follows that location will be the next location to be read.

Therefore, when the second buffer is used and asynchronous reading is performed, a problem occurs in which decoding ends up getting performed at the read starting and ending points, and the read data ends up being different from what is expected.

The following two methods can be used to avoid this problem.

(1) Do not perform asynchronous reading when using the second buffer.
(2) To use the second buffer to perform asynchronous reading, call **sceAtracGetLoopStatus**() to check whether the loop playback status has changed before and after performing asynchronous reading. Only if the loop playback status has not changed, use **sceAtracAddStreamData**() to report the addition of data.

Termination Processing

libatrac3plus internally calls the Media Engine's CPU functions and uses semaphores for exclusive processing of the Media Engine's internal resources.
If a thread that is calling libatrac3plus is forcibly deleted by using **sceKernelTerminateThread**() and **sceKernelTerminateDeleteThread**() before the thread transitions to DORMANT state while a libatrac3plus function is being executed, then depending on the timing, the thread may end up getting deleted while it continues to hold internal resources. As a result, there is a risk that another library that is using the Media Engine's CPU functions besides libatrac3plus may no longer work properly.

To terminate a thread, make sure the following actions are taken (the samples that use libatrac3plus are also constructed in this way):

- The side that requests the thread to terminate uses a shared variable or event flag to communicate its request to the thread.

- The thread that is to be terminated periodically monitors this flag to determine whether a termination request was issued. If a request was issued, the thread first verifies that libatrac3plus is not currently running and then it calls **sceKernelExitThread**() to terminate itself.